CASE FILE · TECHVERSE · TEAM MOLECULENS

Chicago Crime Branch

Simple report, tech stack, pipeline and judge Q&A

1,048,575 crime reports · Python analysis · React dashboard · Detective Dot assistant

1. The project in 5 lines

- We took **1,048,575 Chicago police crime reports**.
- We cleaned them with **Python**.
- We found patterns in time, crime type, arrests and place.
- We checked every number automatically.
- We showed it all in a **website** with a map, filters and a helper called **Dot**.

Golden rule: the website never invents numbers. Python calculates everything; the website only draws it.

2. Tech stack

Part	Tool	What it is for
Analysis	Python 3.13	The TECHVERSE rulebook requires Python for analysis
Data handling	pandas, NumPy, PyArrow	Load, clean and group 1 million rows
Statistics	SciPy	Confidence ranges, tests, corrections
Machine learning	scikit-learn	Arrest model, timing groups, daily-count model
Data contracts	Pydantic	Checks every exported file has the right shape
QA pictures	matplotlib	Contact sheet proving every chart renders
Website	React + TypeScript + Vite	The interactive dashboard
Animation	Motion	Counters, tab slides, bars growing, welcome letters
Chart maths	d3-scale / shape / array / interpolate	Scales, curves, colours (charts themselves are hand-built SVG & Canvas)
Fonts	Bricolage Grotesque, Inter, JetBrains Mono	Installed locally, so no internet needed
Testing	Node scripts + esbuild	Check the website's maths against Python

3. Pipeline: start to finish

```
Crime Dataset.xlsx  (never touched)
   |
1 LOAD      read-only, saved as fast parquet
2 REPAIR    fix swapped dates
3 CLEAN     fix bad map points and empty values
4 FEATURES  Time of Day, Season, Violent/Property/Other...
5 ANALYSE   reliable periods, statistics, models, map data
6 EXPORT    JSON files + a compact "cube"
7 VALIDATE  44 automatic checks
   |
export/ folder  -->  8 DASHBOARD draws it
```

1. **Load.** The 150 MB Excel file is read in read-only mode. A SHA-256 fingerprint proves we never changed it.
2. **Repair dates.** Excel swapped month and day on 424,638 dates. We swapped them back. Proof: "date after update" errors fell from 3.3% to 0%, and the monthly chart matched the handout exactly.
3. **Clean.** 16 impossible map points blanked, community area "0" made missing, empty places set to UNKNOWN. **No rows were deleted.**
4. **Features.** Added Time of Day, Season, Violent/Property/Other (handout definitions), hour, weekday, and flags for suspicious timestamps.
5. **Analyse.** Found the reliable periods, then ran statistics and models.
6. **Export.** KPIs, one file per chart, insights, map data, and the cube. TypeScript types are generated automatically.
7. **Validate.** 44 checks: handout numbers reproduced, totals add up, sort orders correct, cube matches raw rows, original file unchanged.
8. **Dashboard.** Draws the export, and re-checks its own maths against Python on startup ("Verified" badge).

Key finding about the data

The file has exactly 1,048,575 rows (Excel's limit), so it is a **slice**. Only two periods are dense: **April 2015 – July 2017** (window A) and **October 2025 – August 2026** (window B). Elsewhere there are only 300–900 records a month, so we make **no long-term trend claims**. Handout KPIs still use the whole dataset, as required.

4. How the website works

- **The cube:** all rows pre-added into about 524,000 combinations (type, district, month, weekday, time of day, arrest, domestic), 0.66 MB compressed. Filters just add up the right rows, so KPIs, charts and map always agree.
- **Four tabs, not one long scroll:** Dashboard (8 KPIs + 10 required charts), Story, Map, Data trust.
- **Map:** each ~550 m cell takes its most common police district. No boundaries are invented. Colour by crimes, arrest rate, violent, domestic or night share.
- **Colour has meaning:** violent red · property violet · other green · arrests amber · domestic magenta · volume blue.
- **Q9** has a second axis because the handout requires it.

- **Detective Dot** is rule-based, not a free-form AI. It answers only from our exported analysis and live calculations, follows your filters, can compare crimes or districts, tell fun facts and explain charts.
- **Look:** true-crime noir: night-city welcome with flashlight, crime-scene tape, hazard stripe, EXHIBIT/CLUE tags.

5. Rulebook presentation checklist

1) Problem / theme understanding

When, what, where and how do crimes happen, and who gets arrested? Plus the handout's 8 KPI cards and 10 charts.

2) Dataset and source

Given by the organisers: an Excel file, 1,048,575 rows, 22 columns. It *appears to be* the City of Chicago "Crimes – 2001 to Present" data. Say "appears to be", not "is", because we could not confirm it.

3) Cleaning and preparation

Swapped dates repaired, bad map points blanked, zeros and empties handled, nothing deleted, new columns added.

4) Analytical approach

Find reliable periods, compare groups fairly, test on unseen months, report how big each difference is.

5) Key visualisations

The 10 required charts, extra charts, the district map and Dot.

6) Major findings

- Summer has about **17% more** crime per day than winter.
- Weekend nights run about **69% above** weekday nights.
- The evening arrest rate looks higher, but **86% of that gap is explained by crime type**.
- District 11's high arrest rate (38.3%) is explained by its heavy narcotics mix (index 0.98).
- *What* the crime is predicts arrest far better than *when* or *where* (AUC 0.86 on unseen months).

7) Conclusion

Crime follows a rhythm of season, weekend and night. Arrest rates depend mostly on the type of crime. Because of data limits, we do not claim long-term trends.

8) Limitations and assumptions

- The file is a slice, not the full data.
- No population numbers, so no "per person" claims.
- 2025–26 has almost no map coordinates.
- Models show links, not causes.

6. Questions judges may ask (rulebook 5.3), with simple answers and "under the hood"

Each answer has the **simple version** first. The grey box underneath is the **technical version**: which tool, which code file and function, what it does, and which method.

A. Dataset selection

> **Why this dataset?**

It was given by the organisers and matches the theme and handout. Totals match: 1,048,575 crimes and 214,846 arrests.

Tech: Excel (.xlsx), 1 sheet, 22 columns. **Code:** `load.py → load_raw()` reads it with `openpyxl` in read-only mode into a pandas table and caches it as parquet (fast reload). **Check:** `validate.py` asserts K1 = 1,048,575 and arrests = 214,846 (handout values).

> **Is the data complete?**

No. It has exactly Excel's row limit, so it is a slice. Only two periods are fully filled.

Method: count records per day; a "dense window" is a run of at least 60 days with at least 200 records a day. **Code:** `windows.py → dense_windows()` finds them; `pipeline.py → run()` trims them to whole months (A = Apr 2015–Jul 2017, B = Oct 2025–Aug 2026). **Shown in:** the coverage heat map on Data trust → Source (`stats_time.py → coverage()`).

> **Did you change the original file?**

No. We only read it. A checksum (digital fingerprint) proves it did not change.

Code: `load.py → fingerprint()` stores SHA-256 + size + modified-time on first read; `validate.py` recomputes and compares at the end. **Why it matters:** proves the "no modified data" rule.

B. Data-cleaning decisions

> **Why did you change the dates?**

Excel read some dates the wrong way round. After fixing, the monthly chart matched the handout exactly and "date after update" errors fell from 3.3% to 0%.

Evidence: 623,937 cells were text like `MM/DD/YYYY hh:mm AM` (day always ≥ 13) while 424,638 were real Excel datetimes (day always ≤ 12, with a flat "month" distribution). Excel parsed ambiguous dates day-first. **Code:** `clean.py → _parse_stamp()` parses text with `pd.to_datetime(format=...)` and, for datetime cells, rebuilds the date with month and day swapped. **Proof tests:** year still equals the Year column (100%); Date ≤ Updated On improves; Q1 reproduces 28 months with Feb 2016 = 18,710.

> **Why did you not delete bad rows?**

Deleting would change totals the handout expects (K1 must be 1,048,575). We fixed or flagged values instead.

Code: `clean.py → clean()` only blanks (sets to missing) impossible coordinates (outside a Chicago bounding box), turns community area 0 into missing, fills empty places with "UNKNOWN", and logs each count into `cleaning_log.json`. Duplicate IDs = 0, full duplicate rows = 0; 13 repeated case numbers are kept (different IDs).

What are placeholder times?

About 5.5% of records are stamped exactly 00:00 or 12:00, often a "time unknown" default. We keep them for handout numbers but also show a version without them: night-time crime is **17.7%** by the handout definition and **15.8%** without them.

Code: `features.py → add_features()` sets `default_time = (minute == 0) & hour in (0, 12)`. `kpis.py → compute_kpis()` reports K7 literal plus `sensitivity_value`. Hour-level charts exclude these records.

Why are so many crimes dated the 1st of a month?

Some types (sex offence, offences involving children, deceptive practice) are heavily over-represented on day 1. It probably means the true date was unknown. We flag it but cannot prove it.

Method: compare each type's share on day 1 with the expected 1 ÷ 30.4 (≈3.3%). Sex offence has 14.4%, a 4.4× lift.
Code: `stats_time.py → first_of_month_artifact()`. This also explains most of the largest daily "spikes" found by `anomaly_table()`.

C. Code

How is your code organised?

Small files, one job each. One command runs everything.

Python: `load → clean → features → kpis / charts → stats_time / stats_arrest / stats_geo → insights → export → validate`, orchestrated by `pipeline.py → run()` and started by `run_all.py`. Shared maths helpers live in `statlib.py`. **Website:** `engine.ts` (maths), `data.ts` (loading), `charts/` (drawing), `components/` (screens), `assistant.ts` + `dot.ts` (Dot).

How do you know your code is right?

44 automatic checks, including: totals add up, sort orders are correct, handout values are reproduced, and the website's maths matches Python's.

Code: `validate.py → run_checks()` (handout anchors, sums, orders, cube-vs-raw on 25 random filters, source fingerprint); `dashboard/scripts/verify.mjs` bundles `engine.ts` with esbuild and compares all KPIs and Q1–Q10 against the Python export; the page repeats a 15-point self-check on load (the "Verified" badge). **Schemas:** `schema_models.py` (Pydantic) validates every exported file and generates the TypeScript types via `tsgen.py`.

Did you hard-code any findings?

No. Every number is computed from the data, and even the sentences are filled in from computed results.

Code: `insights.py → build_insights()` builds each sentence with f-strings from the results dictionary (e.g. the 17% summer swing comes from `bootstrap_ratio()`). The dashboard reads `insights.json`, and Dot recalculates live on the cube.

D. Analytical methods

What is "standardising"?

Comparing arrest rates fairly. Evenings have more drug crimes, which almost always end in arrest. Standardising pretends every time of day has the same crime mix, so we can see whether time itself matters. It barely does.

Method: *direct standardisation*: for each time of day, take the arrest rate within each crime type and re-weight by the overall crime-type mix. Result: raw Night 18.1% → Evening 25.4% (gap 7.3 pts) becomes 21.4% → 22.4% (gap 1.0 pt), so 86% of the gap was crime mix (narcotics are 2.3% of night crimes but 8.1% of evening crimes). **Code:** `stats_arrest.py → tod_standardised()`. This is "Simpson's paradox".

What is the district "index"?

Real arrests ÷ arrests we would expect from that district's crime types. 1.00 means "as expected". District 11 scores 0.98.

Method: *indirect standardisation (SIR)*: expected arrests = sum over each crime of the city-wide arrest rate for its type. 95% range from the exact Poisson (Garwood) interval; p-values corrected for 22 districts with **Benjamini–Hochberg FDR**; a district is flagged only if corrected p < 0.05 and the range excludes 1. **Code:** `stats_arrest.py → district_sir()`, helpers `poisson_ci()` and `bh_fdr()` in `statlib.py`. **Caveat:** a screening tool; it ignores spatial correlation.

Why give ranges (confidence intervals) and effect sizes?

With about a million rows, almost any difference looks "significant". We show **how big** a difference is and **how sure** we are.

Methods used: Wilson interval (rates), exact Poisson interval (counts), bootstrap ratio with 2,000 resamples (summer ÷ winter = 1.17, 95% range 1.14–1.19), Cohen's w (weekday effect = 0.020, a very small effect), Cramér's V (missing coordinates), Kruskal–Wallis (seasons). **Code:** `statlib.py` (`wilson, poisson_ci, bootstrap_ratio, cramers_v`) and `stats_time.py`.

How did you test the arrest model?

Trained on early months, tested on later months it had never seen. Score **AUC 0.86** (0.5 = guessing, 1.0 = perfect).

Method: temporal hold-out: train before 2017-01-01 (474,279 rows), test after (153,508). Models: logistic regression (odds ratios with 15 bootstrap repeats) and `HistGradientBoostingClassifier` (AUC 0.865; logistic 0.836). **Importance:** permutation importance: shuffling crime type costs 0.31 AUC, location 0.10, time of day + weekday only about 0.005. **Code:** `stats_arrest.py → arrest_models()`. Also reports Brier score and a calibration table.

Why only two time windows?

Outside them the file has only 300–900 records a month, so trends would be unreliable.

See the dense-window method above (`dense_windows()`). The "then vs now" comparison uses only calendar months present in both windows (`stats_time.py → era_comparison()`) so seasons don't bias it.

Why is there no per-person crime rate?

The data has no population numbers. We could add them from another cited source later.

Rulebook section 4 allows extra sources only if clearly cited. We kept to the supplied file to avoid mixing sources; this is listed as a limitation and a "what would change the conclusions" item.

E. Visualisations: why this chart?

Chart	Type and why	What it tells	How it is built (tech)
Q1 monthly	**Line**: time is ordered	Summer peak, February dip	`LineChart` in `Charts.tsx`: d3 monotone curve, animated path, crosshair + tooltip, peak/low labels. Data: `engine.compute()` → `monthly[28]`
Q2 weekday	**Columns**, Mon→Sun	Days are close	`Columns`; order fixed by the DAYS array, not alphabetical
Q3 time of day	**Columns**, in order	Afternoon/evening busiest	`Columns`; hour bins Night 0–5, Morning 6–11, Afternoon 12–17, Evening 18–23 (`features.py`)
Q4 season 2016	**Columns**	Summer highest	Cube filtered to year 2016; season looked up from month
Q5 category share	**Donut**	Property ~43%, violent ~29%	`Donut`: SVG circles animated with `strokeDasharray`
Q6 arrest by time	**Columns** (a rate)	Looks higher in the evening	arrests ÷ crimes per time-of-day bin
Q7 top 10 types	**Sorted bars**	Theft, battery lead	`HBars`; colour = crime category; click filters everything
Q8 top 5 domestic	**Sorted bars**	Battery biggest	Same as Q7 using domestic counts
Q9 districts	**Bars + line**, two axes (handout asks)	Busy ≠ high-arrest	`Combo`: left axis crimes, right axis arrest %, click to filter
Q10 heat map	**Heat map**	Weekend nights, weekday afternoons	`Heatmap`: d3 colour ramp, staggered fade-in, hover cell tooltip

Shared functionality: every chart card has a Table view (accessibility), hover tooltips (`showTip()` in `ui.tsx`), keyboard focus, and animations from the `motion` library. Colours follow a validated palette (`dataviz` validator: colour-blind separation and contrast passed).

F. Interpretation

> **Does a high arrest rate mean a district is dangerous?**

No. It describes police outcomes, not danger, and depends heavily on which crimes are reported.

District 11: 38.3% arrested, but its crime mix is 24% narcotics, so the expected rate is 39.5% and the index is 0.98 (`district_sir()`). Location quotient for narcotics there is 3.7 (`stats_geo.py → location_quotients()`).

Does more night crime mean nights are riskier?

Not per hour. Night is only 6 of 24 hours. The exception is weekend nights.

Method: crimes per calendar day by weekday × hour, excluding placeholder times (`stats_time.py → hour_dow_matrix()`). Weekend 00:00–05:59 is 1.69× a weekday's.

Is crime falling?

Window B (2025–26) has about 14% fewer crimes a day than window A (2015–17), but they are two periods about 10 years apart, so we do not call it a trend. Recent arrest data may also be incomplete.

Method: seasonally matched months, bootstrap ratio 0.86 (95% range 0.85–0.87). **Code:** `era_comparison()` . Arrest rate 21.6% → 15.4% is likely understated because later arrests are not yet recorded.

G. Conclusions

Your one-line conclusion?

Crime follows predictable rhythms (summer, weekends, nights), and arrests depend mostly on the type of crime, not time or place.

What next?

Add population data (cited), get the full dataset, and find which dates were estimated.

H. Questions about the website and Dot

How do the filters change every chart so fast?

The website holds a compact "cube" of pre-added counts and just adds up the rows you selected.

Python: `pipeline.py → build_cube()` groups 1M rows into ~524k combinations of type, district, month, weekday, time of day, arrest, domestic (dictionary-coded integer arrays, 0.66 MB gzip). **Browser:** `data.ts → loadBundle()` downloads it; `engine.ts → parseCube()` un-gzips with `DecompressionStream` ; `Engine.compute(filters)` makes one pass over typed arrays (Int32Array) and fills all KPI and chart numbers at once, so everything always agrees.

How does the map work?

Each ~550 m square of the city takes its most common police district, and is coloured by that district's score.

Python: `stats_geo.py → grid_density()` bins latitude/longitude into 0.005° cells and finds each cell's dominant district. **Browser:** `MapPanel.tsx` draws cells on an HTML canvas with `requestAnimationFrame` ; `Engine.districtStats()` gives per-district totals that follow your filters; hover/click uses nearest-cell hit testing. Districts with fewer than 0.15% of crimes are left out of rate rankings (tiny-sample noise).

How does Dot answer questions? Is it AI?

Dot is a rule-based analyst, not a free-form AI. It answers only from our exported analysis and live calculations.

Code: `assistant.ts → answer()` normalises the question, then tries (1) `dot.ts → extra()` : greetings, fun facts, comparisons, chart help, rankings; (2) district / crime-type detection and live calculations on the cube; (3) keyword scoring over a knowledge base built from `insights.json`, `kpis.json`, `methodology.json` and `cleaning_log.json`; (4) an honest "I can't answer that" fallback. `dot.ts → pickChips()` rotates suggestions using a seeded shuffle; `flavour()` varies the wording. Because it never invents numbers, every answer is grounded.

How is the page animated without being slow?

Animations use cheap effects and the heavy maths is done once.

Tech: `motion` for spring/tween animation, CSS transforms for the skyline and tape (GPU-friendly), canvas for the map and welcome backdrop, and one SVG path per skyline layer (not thousands of shapes). Large blur filters were removed after profiling showed they slowed rendering.

7. Code walkthrough: file by file

Python (analysis)

File	What it does (simple)	Key functions / method
`config.py`	Paths and handout definitions in one place	Violent / Property lists, trend window, orders
`load.py`	Reads the Excel file safely, caches it	`load_raw()`, `fingerprint()`
`clean.py`	Repairs dates and cleans values, logs every change	`_parse_stamp()`, `clean()`
`features.py`	Adds Time of Day, Season, category, hour, flags	`add_features()` (pandas `pd.cut`, `isin`)
`windows.py`	Finds the dense periods from daily counts	`dense_windows()`
`kpis.py`	The 8 handout KPIs + sensitivity values	`compute_kpis()`
`charts.py`	The 10 handout charts as tidy, sorted data	`build_handout_charts()`
`statlib.py`	Small, checkable statistics helpers	Wilson, Poisson interval, BH-FDR, bootstrap, Cramér's V
`stats_time.py`	Time analysis: weekday, hour, seasons, anomalies, then-vs-now, timing groups	`day_of_week_index, seasonality, daily_glm, anomaly_table, era_comparison, type_hour_clusters, first_of_month_artifact`
`stats_arrest.py`	Arrest analysis and models	`tod_standardised, district_sir, arrest_models, domestic_vs_nondomestic`
`stats_geo.py`	Map grid, district specialisation, concentration	`grid_density, location_quotients, concentration`

`insights.py`	Turns results into sentences, with no hard-coded numbers	`build_insights()`
`pipeline.py`	Runs all analyses, builds the cube	`run()`, `build_cube()`
`schema_models.py`, `tsgen.py`	Defines and checks the shape of every export; makes TypeScript types	Pydantic models → JSON Schema → `types.ts`
`export.py`	Writes the JSON files, manifest (with checksums), data dictionary	`write_all()`
`validate.py`	44 automatic checks	`run_checks()`
`reports.py`, `docs.py`	QA contact sheet and generated documentation	matplotlib
`run_all.py`	One command for everything	load → analyse → export → validate

Website (React + TypeScript)

File	What it does (simple)
`lib/engine.ts`	The calculator: filters in, every KPI/chart number out (`compute`, `districtStats`, `typeStats`)
`lib/data.ts`	Downloads the export files in parallel and builds the cube engine
`lib/ui.tsx`	Number formatting, animated counter, tooltip, resize helper, palette
`charts/Charts.tsx`	All charts drawn as SVG: line, columns, bars, donut, combo, heat map, scatter
`components/MapPanel.tsx`	District map on canvas + ranked list + detail panel
`components/Parts.tsx`	Chart card, KPI card (3D tilt), filter bar, multi-select, insight blocks
`components/TrustPanel.tsx`	Data trust: source, pipeline, metrics, methods, quality, limits
`components/Welcome.tsx`	Noir welcome: skyline, tape, flashlight, letters
`components/Chat.tsx`	Dot's chat window
`lib/assistant.ts`, `lib/dot.ts`	Dot's brain: question matching, live answers, fun facts, comparisons, rotating suggestions
`App.tsx`	Puts it together: tabs, filters, the self-check badge
`scripts/sync-data.mjs`, `verify.mjs`	Copies the export into the site; proves browser maths = Python maths

How data flows (one example: clicking "Theft")

1. You click the Theft bar in Q7 (`HBars` → `pickType()` in `App.tsx`).
2. The filter state now contains Theft's index.
3. `Engine.compute()` loops the 524k cube rows once, keeping only Theft rows.

4. All 8 KPI cards, 10 charts and the map recalculate and animate to the new values (Theft total = 236,314, matching the Q7 bar).
5. Dot also answers using the same filters.

8. Methods in plain words

1. **Find reliable periods:** count crimes per day; keep long stretches with plenty of data.
2. **Compare fairly:** standardise, give ranges, correct for testing many things at once.
3. **Test on unseen data:** train on earlier months, test on later ones.
4. **Spot odd days:** predict normal daily crime from weekday, month and holiday; flag big misses.
5. **Group crimes by timing:** cluster by 24-hour pattern (groups are loose, not sharp).
6. **Concentration:** crime is fairly spread out; the busiest 10% of police beats hold about 18% of crime.

9. Rulebook rules and how we follow them

Rule	Our answer
Use Python	All analysis is in Python. The website only draws results.
State the data source	Say it in the presentation (section 5, item 2).
Don't fabricate data	Every number is computed; 44 automatic checks.
Explain assumptions	A "Data trust" tab lists them all.
No pre-made project	Everything was built during this session.
Everyone can explain their part	Use sections 6, 7 and 8 as your cheat-sheet.

10. Judging marks and where we earn them

Criteria (marks)	What we show
Understanding the theme (10)	The questions in section 5.1
Data quality and relevance (15)	Source, coverage heat map, the "slice" finding
Cleaning (10)	Date repair and the cleaning log
Analytical approach (20)	Standardising, district index, held-out model, anomalies
Visualisation (15)	10 required charts, map, meaningful colour, tooltips, filters
Insights (15)	The findings with caveats
Technical (10)	One-command pipeline, validation, type-checked website
Presentation (5)	Welcome page, story tabs, Dot

11. Do's and don'ts

- **Do** say "appears to be from the City of Chicago portal".

- **Do** mention the date swap. It shows you inspected the data.
- **Don't** say "crime is falling" or "this district is dangerous".
- **Don't** say Dot is an AI that knows everything. It answers only from our data.
- **Do** let every team member practise one section.

12. Honest limitations

- File is a slice, so no long-term trend claims. No population data, so no per-capita claims.
- Window B has almost no coordinates, and its arrest rate is probably understated.
- Models show association, not cause. Dot is rule-based.
- Only desktop sizes were tested. The pytest wrapper was not run (pytest not installed); the same 44 checks run inside run_all.py.

13. Folder map and how to run

```
techverse_analysis/
 |- run_all.py     runs the whole analysis + checks
 |- src/           Python code
 |- data/          cached and cleaned data
 |- export/        JSON the website reads
 |- reports/       analysis report + chart contact sheet
 |- dashboard/     the React website  (npm run dev)
```

Website: open cmd, go to the **dashboard** folder, run **npm install** then **npm run dev**, and open **http://localhost:5173**. Redo the analysis with **python run_all.py** in the parent folder.